Lab 5: Introduction to Stacks and Queues Using OOPS

1 Introduction to Stacks and Queues

In this lab, we will explore the concepts of stacks and queues. Both are fundamental data structures in computer science, widely used in various applications.

1.1 Stacks: A stack is a linear data structure that follows the Last In, First Out (LIFO) principle. The operations commonly associated with stacks are:

- `push()` - Add an element to the top of the stack.
- `pop()` - Remove and return the top element from the stack.
- `peek()` - Return the top element without removing it.
- `isEmpty()` - Check if the stack is empty.

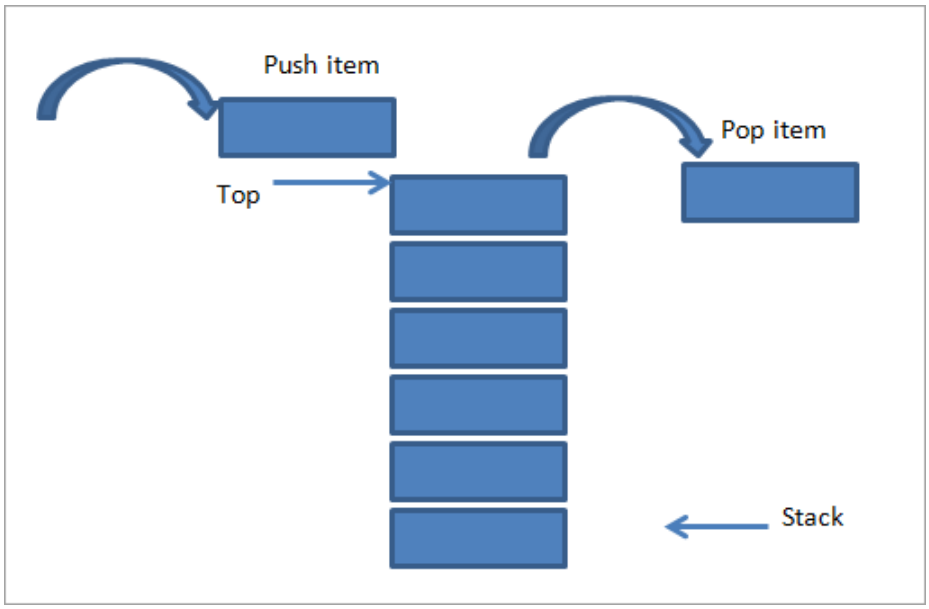

Figure 1: Class example

1.1.1 Stack Implementation in Python

We will implement a stack using a Python class. This custom implementation will help you understand how stacks work internally.

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
```

PREPARED BY: Sahil Wagh (M.Tech)

```
        self.stack.append(item)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return None

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        return None

    def is_empty(self):
        return len(self.stack) == 0
```

Listing 1: Custom Stack Implementation

1.1.2 Detailed Explanation of Stack Methods

- `__init__()` - Initializes the stack. The stack is represented by a list, `self.stack`, which starts empty.
- `push(item)` - Adds an item to the top of the stack. Internally, it appends the item to the list.
- `pop()` - Removes the top item from the stack and returns it. If the stack is empty, it returns `None`.
- `peek()` - Returns the top item from the stack without removing it. If the stack is empty, it returns `None`.
- `is_empty()` - Checks whether the stack is empty. It returns `True` if the stack is empty, otherwise `False`.

1.1.3 Dry Run and Output

Let's perform a dry run using the following sequence of operations:

Step 1: Create a stack object.

```
my_stack = Stack()  # Creates an empty stack
```

Step 2: Push elements onto the stack.

```
my_stack.push('a')  # Stack: ['a']
my_stack.push('b')  # Stack: ['a', 'b']
my_stack.push('c')  # Stack: ['a', 'b', 'c']
my_stack.push('d')  # Stack: ['a', 'b', 'c', 'd']
```

Step 3: Peek at the top element of the stack.

```
top_element = my_stack.peek()  # Returns 'd'
```

Step 4: Pop the top element from the stack.

```
popped_element = my_stack.pop()  # Returns 'd', Stack: ['a', 'b', 'c']
```

Step 5: Pop another element from the stack.

```
popped_element = my_stack.pop()  # Returns 'c', Stack: ['a', 'b']
```

Step 6: Check if the stack is empty.

PREPARED BY: Sahil Wagh (M.Tech)

```
is_empty = my_stack.is_empty()  # Returns False
```

Output:

```
'd'
'c'
False
```

1.2 Queues: A queue is a linear data structure that follows the First In, First Out (FIFO) principle. The operations commonly associated with queues are:

- `enqueue()` - Add an element to the end of the queue.
- `dequeue()` - Remove and return the front element from the queue.
- `peek()` - Return the front element without removing it.
- `isEmpty()` - Check if the queue is empty.

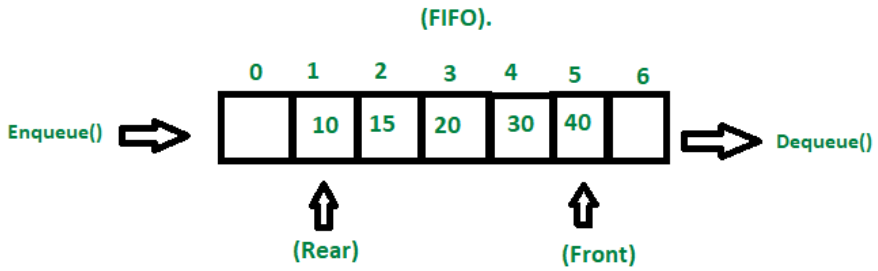

Figure 2: queue

1.2.1 Queue Implementation in Python

We will implement a queue using a Python class. This custom implementation will help you understand how queues work internally.

```
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        return None

    def peek(self):
        if not self.is_empty():
            return self.queue[0]
        return None

    def is_empty(self):
        return len(self.queue) == 0
```

Listing 2: Custom Queue Implementation

1.2.2 Detailed Explanation of Queue Methods

- `__init__()` - Initializes the queue. The queue is represented by a list, `self.queue`, which starts empty.
- `enqueue(item)` - Adds an item to the end of the queue. Internally, it appends the item to the list.
- `dequeue()` - Removes the front item from the queue and returns it. If the queue is empty, it returns `None`.
- `peek()` - Returns the front item from the queue without removing it. If the queue is empty, it returns `None`.
- `is_empty()` - Checks whether the queue is empty. It returns `True` if the queue is empty, otherwise `False`.

1.2.3 Dry Run and Output

Let's perform a dry run using the following sequence of operations:

Step 1: Create a queue object.

```
my_queue = Queue()  # Creates an empty queue
```

Step 2: Enqueue elements into the queue.

```
my_queue.enqueue('a')  # Queue: ['a']
my_queue.enqueue('b')  # Queue: ['a', 'b']
my_queue.enqueue('c')  # Queue: ['a', 'b', 'c']
my_queue.enqueue('d')  # Queue: ['a', 'b', 'c', 'd']
```

Step 3: Peek at the front element of the queue.

```
front_element = my_queue.peek()  # Returns 'a'
```

Step 4: Dequeue the front element from the queue.

```
dequeued_element = my_queue.dequeue()  # Returns 'a', Queue: ['b', 'c', 'd']
```

Step 5: Dequeue another element from the queue.

```
dequeued_element = my_queue.dequeue()  # Returns 'b', Queue: ['c', 'd']
```

Step 6: Check if the queue is empty.

```
is_empty = my_queue.is_empty()  # Returns False
```

Output:

```
'd'
'c'
False
```

2 Implementing Stack using `collections.deque`

The Python `collections.deque` class provides an efficient way to implement a stack. The `deque` (double-ended queue) allows us to add and remove elements from both ends with $O(1)$ time complexity.

2.0.1 Code Example

```
from collections import deque

stack = deque()

# Push elements onto the stack
stack.append('a')
stack.append('b')
stack.append('c')

# Pop elements from the stack
top_element = stack.pop()  # 'c'
next_element = stack.pop()  # 'b'

# Check the top element
peek_element = stack[-1]  # 'a'

# Check if the stack is empty
is_empty = len(stack) == 0  # False
```

Listing 3: Stack Implementation using deque

2.0.2 Dry Run and Output

Let's perform a dry run of the above code:

Step 1: Create a stack using `deque`.

```
stack = deque()  # Creates an empty stack
```

Step 2: Push elements onto the stack.

```
stack.append('a')  # Stack: deque(['a'])
stack.append('b')  # Stack: deque(['a', 'b'])
stack.append('c')  # Stack: deque(['a', 'b', 'c'])
```

Step 3: Pop elements from the stack.

```
top_element = stack.pop()  # 'c', Stack: deque(['a', 'b'])
next_element = stack.pop()  # 'b', Stack: deque(['a'])
```

Step 4: Check the top element.

```
peek_element = stack[-1]  # 'a'
```

Step 5: Check if the stack is empty.

```
is_empty = len(stack) == 0  # False
```

Output:

```
'c'
'b'
'a'
False
```

3 Implementing Queue using `collections.deque`

Similar to stacks, the `deque` class can be used to implement queues efficiently. We use `append()` to enqueue elements and `popleft()` to dequeue elements.

PREPARED BY: Sahil Wagh (M.Tech)

3.0.1 Code Example

```
from collections import deque

queue = deque()

# Enqueue elements into the queue
queue.append('a')
queue.append('b')
queue.append('c')

# Dequeue elements from the queue
front_element = queue.popleft()  # 'a'
next_element = queue.popleft()  # 'b'

# Check the front element
peek_element = queue[0]  # 'c'

# Check if the queue is empty
is_empty = len(queue) == 0  # False
```

Listing 4: Queue Implementation using deque

3.0.2 Dry Run and Output

Let's perform a dry run of the above code:

Step 1: Create a queue using `deque`.

```
queue = deque()  # Creates an empty queue
```

Step 2: Enqueue elements into the queue.

```
queue.append('a')  # Queue: deque(['a'])
queue.append('b')  # Queue: deque(['a', 'b'])
queue.append('c')  # Queue: deque(['a', 'b', 'c'])
```

Step 3: Dequeue elements from the queue.

```
front_element = queue.popleft()  # 'a', Queue: deque(['b', 'c'])
next_element = queue.popleft()  # 'b', Queue: deque(['c'])
```

Step 4: Check the front element.

```
peek_element = queue[0]  # 'c'
```

Step 5: Check if the queue is empty.

```
is_empty = len(queue) == 0  # False
```

Output:

```
'a'
'b'
'c'
False
```